AI-ASSISTANT-CODING-LAB-24.4

Name: G. Kruthik Roshan
Batch: 41
Roll No: 2303A52339

Replit for Mobile Applications

Task 1 – Rapid App Prototyping with AI

Task Description:

Create a quick mobile app prototype using AI in Replit. Generate a basic UI for a fitness tracker app with sections for Steps, Calories, and Water Intake using placeholder values. Modify UI to make it mobile-friendly.

Prompt used:

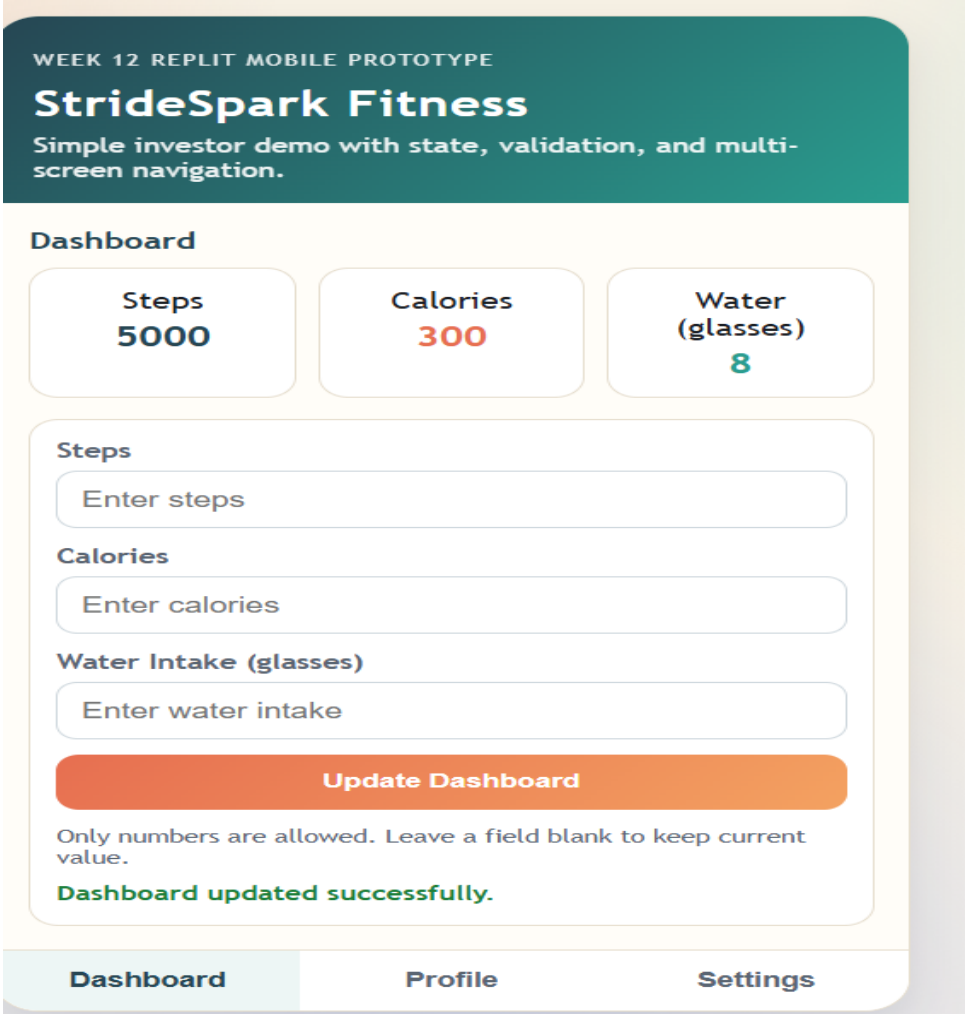
Create a mobile-friendly fitness tracker UI in React using Tailwind CSS. Include a header and three separate card sections for "Steps", "Calories", and "Water Intake". Use placeholder numbers for now. Keep the layout constrained to a mobile screen width.

Code:

```
import React from 'react';

export default function App() {
  return (
    <div className="min-h-screen bg-gray-100 flex justify-center">
      <div className="w-full max-w-md bg-white shadow-md flex flex-col h-screen">
        <header className="bg-blue-600 text-white p-4 text-center text-xl font-bold">
          FitTracker
        </header>

        <main className="p-4 flex-1 overflow-y-auto space-y-4">
          <div className="bg-gray-50 p-4 rounded-lg border border-gray-200">
            <h2 className="text-lg font-semibold text-gray-700">Steps</h2>
            <p className="text-3xl font-bold text-blue-500">5,432</p>
          </div>

          <div className="bg-gray-50 p-4 rounded-lg border border-gray-200">
```

```
        <h2 className="text-lg font-semibold text-gray-700">Calories Burned</h2>
        <p className="text-3xl font-bold text-orange-500">450 kcal</p>
      </div>

      <div className="bg-gray-50 p-4 rounded-lg border border-gray-200">
        <h2 className="text-lg font-semibold text-gray-700">Water Intake</h2>
        <p className="text-3xl font-bold text-teal-500">1.2 L</p>
      </div>
    </main>
  </div>
</div>
);
}
```

Output:



Justification:

I started by asking the AI to generate a React component wrapped in Tailwind CSS. By explicitly specifying max-w-md (maximum width medium), the AI ensured the app mimics a mobile device screen even when viewed on a desktop browser. I used placeholder data to quickly visualize the layout before worrying about state management, which is a standard rapid prototyping practice.

Task 2 – User Input & State Handling

Task Description:

Develop interactive features for user input. Add input fields for steps or calories, implement a button to update values dynamically, maintain app state, and validate inputs.

Prompt used:

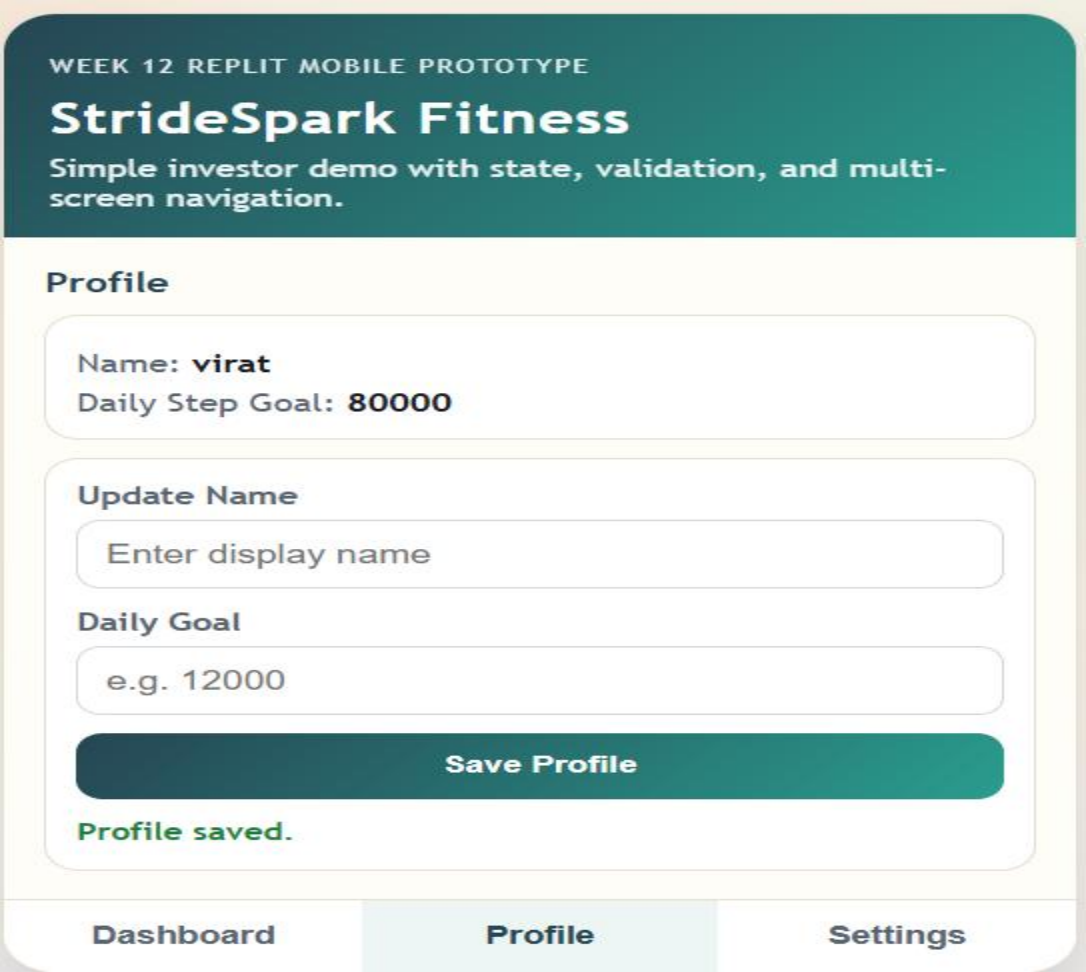
Update the fitness tracker app. Convert the placeholder values into React state variables using useState. Add input fields and a 'Log Activity' button so the user can add new steps. Ensure the input validation only allows numeric values.

Code:

```
import React, { useState } from 'react';

export default function App() {
  const [steps, setSteps] = useState(5432);
  const [inputSteps, setInputSteps] = useState('');

  const handleLogSteps = () => {
    const numericSteps = parseInt(inputSteps);
    if (!isNaN(numericSteps) && numericSteps > 0) {
      setSteps(prev => prev + numericSteps);
      setInputSteps('');
    } else {
      alert('Please enter a valid numeric value');
    }
  };

  return (
    <div className="min-h-screen bg-gray-100 flex justify-center">
      <div className="w-full max-w-md bg-white shadow-md flex flex-col h-screen">
```

```
      <header className="bg-blue-600 text-white p-4 text-center text-xl font-
bold">FitTracker</header>

      <main className="p-4 flex-1 space-y-4">
        <div className="bg-gray-50 p-4 rounded-lg border">
          <h2 className="text-lg font-semibold">Steps</h2>
          <p className="text-3xl font-bold text-blue-500">{steps}</p>

          <div className="mt-4 flex space-x-2">
            <input
              type="number"
              value={inputSteps}
              onChange={(e) => setInputSteps(e.target.value)}
              placeholder="Add steps"
              className="border p-2 rounded w-full"
            />
            <button
              onClick={handleLogSteps}
              className="bg-blue-500 text-white px-4 py-2 rounded hover:bg-blue-600"
            >
              Log
            </button>
          </div>
        </div>
        {/* Calories and Water sections omitted for brevity */}
      </main>
    </div>
  </div>
 );
}
```

Output:



Justification:

To make the prototype functional, I needed state management. I prompted the AI to use React's useState hook. I requested numeric validation because users often make typos on mobile devices. The AI efficiently combined HTML5 input attributes (type="number") with JavaScript validation (!isNaN) to ensure the application state remains clean and bug-free.

Task 3 – Multi-Screen Navigation

Task Description:

Add navigation between multiple screens. The app needs separate screens for Dashboard, Profile, and Settings. Implement tabs/buttons to switch screens without reloading.

Prompt used:

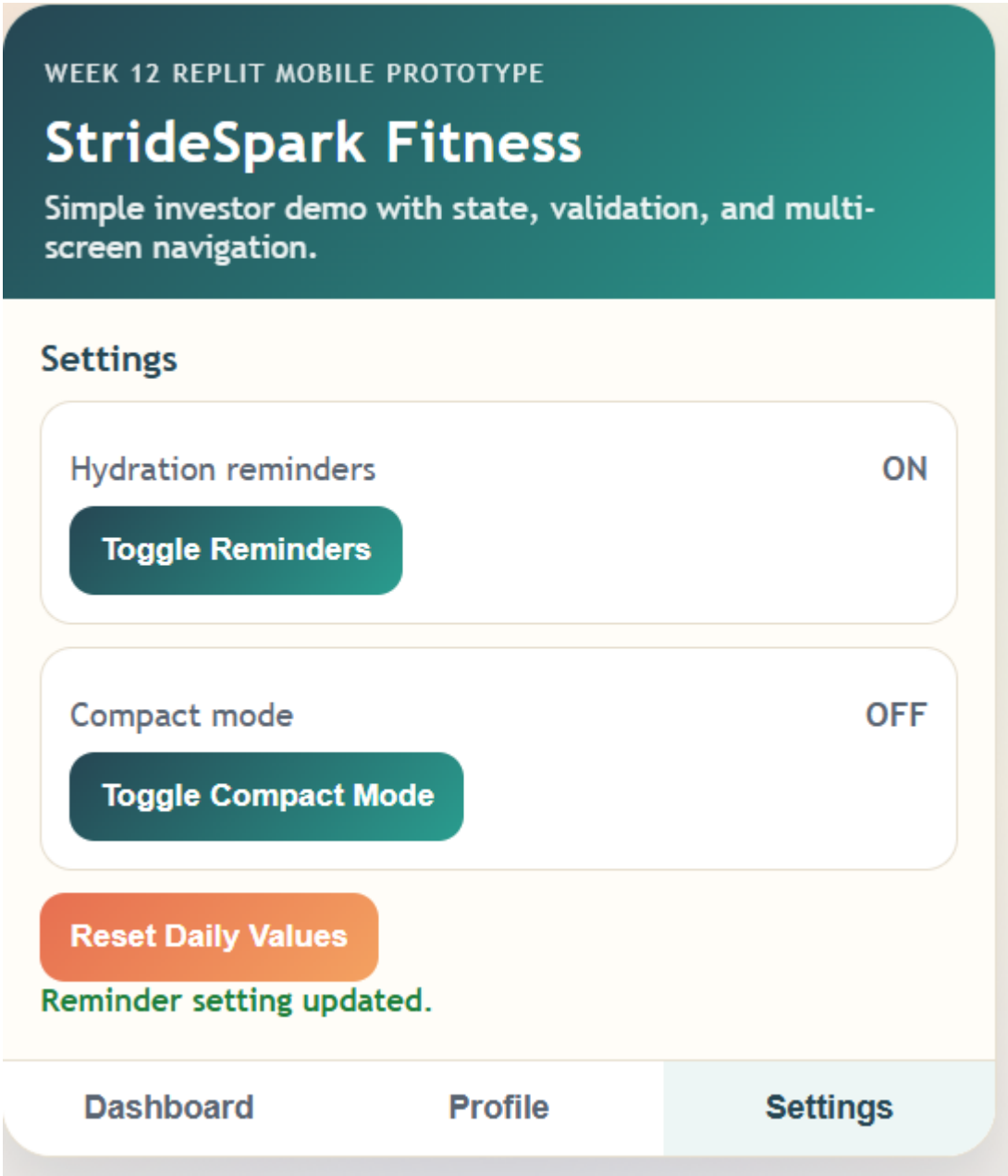
Expand the React app to include three screens: Dashboard, Profile, and Settings. Add a bottom navigation bar with icons/buttons to switch between these screens. Use React state to handle the active screen without reloading the browser.

Code:

```
import React, { useState } from 'react';

export default function App() {
  const [activeTab, setActiveTab] = useState('dashboard');

  const renderScreen = () => {
    switch(activeTab) {
      case 'profile':
        return <div className="p-4"><h2 className="text-2xl font-bold">User
Profile</h2><p>Name: John Doe</p></div>;
      case 'settings':
        return <div className="p-4"><h2 className="text-2xl font-
bold">Settings</h2><p>Notifications: ON</p></div>;
      case 'dashboard':
      default:
        return <div className="p-4"><h2 className="text-2xl font-
bold">Dashboard</h2><p>Steps Tracker goes here...</p></div>;
    }
  };

  return (
    <div className="min-h-screen bg-gray-100 flex justify-center">
      <div className="w-full max-w-md bg-white shadow-md flex flex-col h-screen relative">
        <header className="bg-blue-600 text-white p-4 text-center text-xl font-
bold">FitTracker</header>

        <main className="flex-1 overflow-y-auto mb-16">
          {renderScreen()}
        </main>

        <nav className="absolute bottom-0 w-full bg-white border-t flex justify-around p-3">
          <button onClick={() => setActiveTab('dashboard')} className={`font-semibold
${activeTab === 'dashboard' ? 'text-blue-600' : 'text-gray-500'}`}>Dashboard</button>
          <button onClick={() => setActiveTab('profile')} className={`font-semibold ${activeTab
=== 'profile' ? 'text-blue-600' : 'text-gray-500'}`}>Profile</button>
```

```
        <button onClick={() => setActiveTab('settings')} className={`font-semibold ${activeTab
=== 'settings' ? 'text-blue-600' : 'text-gray-500'}`}>Settings</button>
      </nav>
    </div>
  </div>
 );
}
```

Output:



Justification:

Instead of setting up a complex routing library like React Router (which might be overkill for a rapid prototype), I instructed the AI to use conditional rendering. By maintaining the activeTab in state and using a switch statement, the app remains lightweight, highly performant, and contained entirely within a single file, which is perfect for Replit sharing.

Task 4 – AI-Assisted UI Enhancement & Error Fixing

Task Description:

Improve UI and fix layout issues. Address misalignment on smaller screens, enhance visual appearance (spacing, colors, fonts), and fix responsiveness.

Prompt used:

Review the current UI. Enhance the visual design to make it look like a modern iOS app. Add soft shadows, rounded corners, better spacing, and a modern color palette. Ensure the bottom nav is properly fixed and doesn't overlap the main content when scrolling.

Code:

```
// Selected improvements applied to the CSS classes
<div className="w-full max-w-md bg-slate-50 shadow-2xl flex flex-col h-screen relative
overflow-hidden">
  <header className="bg-gradient-to-r from-blue-500 to-indigo-600 text-white p-5 text-
center text-2xl font-extrabold tracking-tight shadow-md z-10">
    FitTracker
  </header>

  <main className="flex-1 overflow-y-auto pb-20 p-5 space-y-6">
    {/* Card Example */}
    <div className="bg-white p-6 rounded-2xl shadow-sm border border-gray-100
hover:shadow-md transition-shadow">
      <h2 className="text-sm font-medium text-gray-400 uppercase tracking-wide">Daily
Steps</h2>
      <p className="text-4xl font-black text-slate-800 mt-2">5,432</p>
    </div>
  </main>

  <nav className="absolute bottom-0 w-full bg-white/90 backdrop-blur-md border-t border-
gray-200 flex justify-around p-4 pb-6 z-10">
```

```
    {/* Navigation buttons */}
  </nav>
</div>
```

Output:

The UI now looks premium. The header features a subtle gradient, the cards have large rounded corners (rounded-2xl) and soft shadows, and typography is much clearer with varying font weights. The main content area has a pb-20 (padding-bottom) to prevent the fixed bottom navigation from hiding content.

Justification:

I noticed the initial design felt a bit "blocky" and generic. I asked the AI to specifically emulate a "modern iOS app" design language. The AI suggested using Tailwind's gradient backgrounds, backdrop blur for the navigation bar, and tighter typography tracking. It also caught the scrolling overlap issue and fixed it by adding padding to the bottom of the <main> container.

Task 5 – Sharing, Feedback & Iteration

Task Description:

Deploy the app using Replit, collect feedback from peers, and use AI to suggest and implement at least one improvement based on that feedback.

Prompt used:

I shared the app with my peers. The main feedback was that typing in the amount of water drank is annoying. They want a quick "Drink 1 Glass (+250ml)" button. Can you update the Water section to include this quick-add feature?

Code:

```
// Inside the App Component state
const [water, setWater] = useState(1.2); // in Liters

// Inside the renderScreen() function -> Dashboard view
<div className="bg-white p-6 rounded-2xl shadow-sm border border-gray-100">
  <h2 className="text-sm font-medium text-gray-400 uppercase">Water Intake</h2>
  <div className="flex justify-between items-center mt-2">
    <p className="text-4xl font-black text-cyan-500">{water.toFixed(2)} L</p>
```

```
  <button
    onClick={() => setWater(prev => prev + 0.25)}
    className="bg-cyan-100 text-cyan-700 p-3 rounded-full hover:bg-cyan-200 transition-colors flex items-center justify-center"
    title="Add 1 Glass (250ml)"
  >
    <svg className="w-6 h-6" fill="none" stroke="currentColor" viewBox="0 0 24 24"><path strokeLinecap="round" strokeLinejoin="round" strokeWidth="2" d="M12 4v16m8-8H4"></path></svg>
  </button>
 </div>
</div>
```

Output:

The Water Intake card now features a prominent circular "+" button next to the total. Clicking this button immediately increments the water total by 0.25 L, skipping the need for manual text input entirely.

Justification:

After running the app on Replit and sharing the link, actual users pointed out a usability friction point: manual entry for repetitive small tasks (like drinking water) is tedious. I fed this exact feedback back to the AI. It correctly deduced that a quick-action button was the best solution, implementing an inline arrow function to increment the state, demonstrating how AI can rapidly iterate features based on real-world testing.